DAL-585
AY 2026-27 Autumn Semester
Programming Assignment 2

## Due Date

Friday 29 October 2026 at 5pm. Late assignments will not be accepted and will receive ZERO mark.

## Objective

In this assignment, you are to write a program (in Python v3) to perform part- of-speech (POS) tagging. Given a sentence, the task is to assign a POS tag to each word in the sentence. We will adopt the Penn Treebank tag set in this assignment.

For example, given the following input sentence:

```
He also is a consensus manager .
```

The POS tagger should output the following:

```
He/PRP also/RB is/VBZ a/DT consensus/NN
manager/NN ./.
```

Note that each input sentence has been tokenized (punctuation symbols are separated from words).

**You are to implement a POS tagger using word vectors and LSTM network. DO NOT use any kind of pre-trained word vectors. Total dimension of the word vectors must not cross 100.**

## Training and Testing

You are provided with a training set of POS-tagged sentences (`train.tags`). You will train your POS tagger using this training set.

The command for training (and if necessary tuning) the POS tagger is:

```
Python3 build_tagger.py train.tags
```

Save your files generated by training process in the current working directory (CWD). You need to use this files during inference process. You can use some part of the training data for hyper-parameter settings.

A separate development set of sentences is also provided (`dev.sents`) along with its POS tags (dev.tags). You use these two files to measure the performance of your POS tagger.

The command to test and generate an output file is:

```
Python3 run_tagger.py dev.sents dev.out
```

The output file dev.out has the same format as the POS-tagged training file. A sample output file is also provided (sample_test`.out`). Generate the dev.out in CWD.

To measure the performance of your tagger, you can use the MeasurePOSTaggerAccuracy.py file. The command for this is as follows:

```
Python3 MeasurePOSTaggerAccuracy.py dev.out dev.tags
```

First argument of this command must be your output file and second argument must be the gold-label POS tag file.

The commands `build_tagger.py` and `run_tagger.py` as shown above will be executed to evaluate your POS tagger, except that testing will be done on a set of new, blind test sentences. The blind test file consists of a list of sentences (without POS tags), one sentence per line. A sample test file is provided (sample_`test.sents`).

## Do not use hard coded file paths in your code.

Ideally execution time of build_tagger.py and run_tagger.py should not cross 15 min and 5 min, respectively. Your training and test code will be run on Google Colab. So please test your code on Colab to make sure it can finish the training and inference in time. Use following environment in Colab:

Python 3.10

PyTorch 2.11+cu128

NumPy 2.1.3

**Deliverables**

You are to work on this programming assignment *in group of two (three for few cases as approved by instructor)*. Please submit the group names, members and group leader to CR Rahil. You must hand in the following:

1. Your source code (with documentation if needed). Note that graphical user interface is not needed. Code must be submitted via the following google form as single zip file. Format of the zip file name is DAL585_PA2_<RollNumber>_<Name>.zip. **Only group leader must submit the code.**

   https://forms.gle/LbRrhKzQFC5UmuvSA

2. A **hard copy** of short report in PDF format (4 to 6 pages) describing details of your implemented method. The report must be prepared on overleaf.com using ACL format. TA will help you with this.

**Content of the Report**

1. Group name, name and roll number of the group members must be mentioned along with the title of the report.
2. It should have sections such as Abstract, Introduction, Model Architecture, Training Algorithms, Loss Functions, Results, Error Analysis, Conclusion. You must describe how unknown tokens during test time are handled. How did you prepare the vocabulary, etc. This report must be structured in a coherent fashion like a research paper.
3. Your report must include a clean architecture diagram of your neural model (use https://www.drawio.com/ for this) and describe the model from input to output like a research paper. You must include the transformation that happens from input to output using standard notations used in NLP literatures.
4. You must include the details of training process such as batch size, learning rate, learning algorithms, number of trainable parameters. It must include the running time of the build_tagger.py and

run_tagger.py. Put all such information in the table format.

5. Include a result table in the report that should have a baseline score on the dev.sents obtained in the first assignment using HMM model and performance of this neural tagger. Your report must include error analysis of this neural tagger agains the HMM tagger.
6. Above content is just a minimum things that must be there in the report. You must include any other important information to make your report distinctive from others.

**Grading**

6*0%* for your report.
4*0%* for the accuracy of your POS tagger, as measured on the blind test set of sentences.